บทที่ 11

การใช้งานกล้อง อัลบั้มภาพ และระบบจัดเก็บไฟล์เนทีฟร่วมกับคลาวด์

การสร้างสรรค์โมบายแอปพลิเคชันที่ต้องทำงานกับรูปภาพ ไม่เพียงแค่การถ่ายรูปหรือดึงภาพมาแสดงบนหน้าจอเท่านั้น แต่นักพัฒนาจำเป็นต้องมีความเข้าใจอย่างลึกซึ้งเกี่ยวกับการจัดการพื้นที่จัดเก็บไฟล์ (File Storage) เพื่อให้ภาพที่ได้ไม่สูญหายและถูกเผยแพร่ในระดับสิทธิ์ที่เหมาะสม ในบทเรียนนี้เราจะเรียนรู้ตั้งแต่การเข้าถึงกล้องและอัลบั้มภาพเนทีฟ ไปจนถึงระบบการจัดการไฟล์ 3 รูปแบบหลัก ได้แก่ ไฟล์เฉพาะแอป (Sandbox), ไฟล์แชร์ร่วมกับแอปอื่น (Shared Gallery) และไฟล์บนคลาวด์ (Supabase Cloud Storage)

11.1 กลไกการจัดการสิทธิ์กล้องถ่ายรูปและคลังภาพ (Camera & Media Permissions)

การเข้าถึงกล้องถ่ายรูปและการสืบค้นรูปภาพเก่าในเครื่องถือเป็นเรื่องละเอียดอ่อนต่อความเป็นส่วนตัว (Privacy) ระบบปฏิบัติการ iOS และ Android จึงมีระบบความปลอดภัยและบังคับร้องขอสิทธิ์แยกประเภทกันอย่างชัดเจน:

1. สิทธิ์การเปิดเลนส์กล้อง (Camera Permission)

สำหรับการใช้ถ่ายรูปสด ๆ ในแอป เราสามารถใช้ Hook `useCameraPermissions()` ของแพ็คเกจ `expo-camera` เพื่อตรวจสอบสถานะสิทธิ์และยื่นขอสิทธิ์ได้โดยตรง:

```
import { useCameraPermissions } from 'expo-camera';

const [permission, requestPermission] = useCameraPermissions();

// ตรวจสอบสถานะการได้รับสิทธิ์
if (!permission) {
  // สถานะยังคงกำลังโหลดเช็ค
}
if (!permission.granted) {
  // สิทธิ์ยังไม่ได้เปิดอนุมัติ ต้องกดเรียก requestPermission()
}
```

2. สิทธิ์การเข้าถึงคลังสื่อเพื่อเลือกรูปภาพ (Media Library / Photos Permission)

สำหรับการดึงภาพถ่ายเดิมจากคลังภาพมาใช้งานผ่าน `expo-image-picker` จะต้องมีการร้องขอสิทธิ์แยกออกต่างหาก:

```
import * as ImagePicker from 'expo-image-picker';

const requestGalleryPermission = async () => {
  const { status } = await
ImagePicker.requestMediaLibraryPermissionsAsync();
  if (status !== 'granted') {
    alert('จำเป็นต้องขอสิทธิ์เข้าถึงคลังภาพเพื่อเลือกรูปอวาตาร์');
  }
};
```

11.2 การใช้งาน CameraView เพื่อพรีวิวภาพสด (Camera Live Preview)

ใน Expo SDK 54 คอมโพเนนต์หลักที่ใช้เปิดช่องเลนส์รับภาพสดคือ `CameraView` (ถูกพัฒนามาทดแทนคอมโพเนนต์ Camera ตัวเดิม) (Expo, 2024)

พร็อพส์หลักที่ใช้ควบคุม <CameraView>:

• `facing`: กำหนดประเภทเลนส์ที่จะใช้เปิด รับค่า 'front' (กล้องหน้า) หรือ 'back' (กล้องหลัง)

• `autofocus`: เปิดหรือปิดระบบดักจับโฟกัสภาพอัตโนมัติ ('on' หรือ 'off')

• `ref`: ตัวเชื่อมโยง (Reference) เพื่อเรียกคำสั่งลั่นชัตเตอร์ถ่ายภาพนิ่ง

```
import React, { useState } from 'react';
import { StyleSheet, Text, View, Button, TouchableOpacity } from 'react-
native';
import { CameraView, CameraType, useCameraPermissions } from 'expo-
camera';

export default function CameraPreviewApp() {
  const [facing, setFacing] = useState<CameraType>('front');
  const [permission, requestPermission] = useCameraPermissions();

  if (!permission) return <View />; // กำลังโหลดเช็คสิทธิ์

  if (!permission.granted) {
    return (
      <View style={styles.container}>
        <Text style={styles.text}>แอปต้องการสิทธิ์เข้าใช้กล้องถ่ายรูป</Text>
        <Button onPress={requestPermission} title="อนุมัติสิทธิ์การเข้าใช้งานกล้อง" />
      </View>
  );
  }

  return (
    <View style={styles.container}>
```

```
      <CameraView style={styles.camera} facing={facing}>
        <View style={styles.buttonContainer}>
          <TouchableOpacity
            style={styles.button}
            onPress={() => setFacing(curr => curr === 'back' ? 'front' :
'back')}
          >
       <Text style={styles.buttonText}>🔄 สลับมุมกล้อง</Text>
          </TouchableOpacity>
        </View>
      </CameraView>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center' },
  camera: { flex: 1 },
 buttonContainer: { flex: 1, justifyContent: 'flex-end', margin: 40 },
  button: { backgroundColor: 'rgba(0,0,0,0.5)', padding: 12, borderRadius: 8,
alignItems: 'center' },
  buttonText: { color: 'white', fontWeight: 'bold' },
  text: { textAlign: 'center', marginBottom: 10 }
});
```

11.3 การลั่นชัตเตอร์ถ่ายภาพนิ่ง (Taking Photos with takePictureAsync)

เพื่อบันทึกรูปภาพพรีวิวเป็นรูปนิ่ง (Snapshot) เราจะอ้างอิง Ref ไปยังคอมโพเนนต์ `<CameraView>` แล้วเรียกใช้เมธอด Asynchronous ชื่อ **`takePictureAsync()`**:

```
import React, { useRef } from 'react';
import { CameraView } from 'expo-camera';
import { TouchableOpacity, Text } from 'react-native';

export default function CaptureComponent() {
  const cameraRef = useRef<CameraView | null>(null);

  const takePhoto = async () => {
    if (cameraRef.current) {
      const result = await cameraRef.current.takePictureAsync({
        quality: 0.8,          // คุณภาพการบีบอัดภาพ (0.0 ถึง 1.0)
        skipProcessing: false // จัดการหมุนทิศทางภาพถ่ายให้อัตโนมัติ
      });
   // ผลลัพธ์จะคืนค่าเป็นอ็อบเจกต์ที่มี URI ไฟล์ภาพชั่วคราวชี้ที่หน่วยความจำแคช
      console.log('ที่อยู่รูปถ่ายชั่วคราว:', result?.uri);
    }
```

```
  };

  return (
    <CameraView ref={cameraRef} style={{ flex: 1 }}>
      <TouchableOpacity onPress={takePhoto} style={{ alignSelf: 'center',
margin: 40 }}>
        <Text style={{ color: 'white', fontSize: 18, fontWeight: 'bold'
}}>📸 ลั่นชัตเตอร์</Text>
      </TouchableOpacity>
    </CameraView>
 );
}
```

11.4 การเลือกรูปภาพและสื่อผ่าน expo-image-picker (Image Library API)

ในการพัฒนาฟีเจอร์ดึงไฟล์ภาพที่มีอยู่แล้วในคลังอัลบั้มของเครื่องขึ้นมาจัดแสดง เราจะใช้ไลบรารีมาตรฐานยอดนิยมคือ `expo-image-picker` (Expo, 2024) ผ่านคำสั่ง Asynchronous ชื่อ `launchImageLibraryAsync()`:

```
import React, { useState } from 'react';
import { View, Button, Image, StyleSheet } from 'react-native';
import * as ImagePicker from 'expo-image-picker';

export default function PickImageApp() {
  const [imageUri, setImageUri] = useState<string | null>(null);

  const pickImage = async () => {
    const result = await ImagePicker.launchImageLibraryAsync({
      mediaTypes: ImagePicker.MediaTypeOptions.Images, // คัดเฉพาะประเภทไฟล์ภาพนิ่ง
      allowsEditing: true, // เปิดกรอบแก้ไข/ตัดรูปภาพหน้าเนทีฟของอุปกรณ์
      aspect: [1, 1],      // บังคับครอปเป็นสัดส่วนจัตุรัส กว้าง:สูง
      quality: 0.8,        // กำหนดความละเอียดผลลัพธ์
    });

    if (!result.canceled && result.assets && result.assets.length > 0) {
      setImageUri(result.assets[0].uri);
    }
  };

  return (
    <View style={styles.container}>
      <Button title="เลือกรูปภาพจากอัลบั้ม" onPress={pickImage} />
      {imageUri && <Image source={{ uri: imageUri }} style={styles.image}
/>}
```

```
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', alignItems: 'center' },
  image: { width: 150, height: 150, borderRadius: 75, marginTop: 15 }
});
```

11.5 ระบบจัดการไฟล์แบบที่ 1: Private Sandbox Storage (แอปมองเห็นเฉพาะตัว)

รูปภาพที่ได้จากการสั่งถ่ายด้วยกล้องถ่ายรูป หรือได้จากการดึงภาพของ Image Picker จะได้รับการสร้างเป็นที่อยู่พาธชั่วคราวอยู่ใน **Cache Directory** เสมอ (Expo, 2024) (เช่น `file:///.../Caches/xxx.jpg`)

ในระบบปฏิบัติการ iOS และ Android พื้นที่ความจำของแอปพลิเคชันจะถูกจำกัดอยู่ในเขตปลอดภัยที่เรียกว่า **Sandbox** โดยแบ่งออกเป็นสองระดับหลัก ๆ:

1. **Cache Folder (`cacheDirectory`)**: ใช้เก็บผลลัพธ์ชั่วคราว มีข้อเสียหลักคือ **OS สามารถกวาดล้างลบไฟล์ออกได้อัตโนมัติ** เมื่ออุปกรณ์ต้องการทวงคืนพื้นที่ดิสก์ว่างคืน หากเปิดแอปขึ้นมาอีกครั้ง ภาพโปรไฟล์ก็อาจชำรุดเสียหายได้
2. **Documents Folder (`documentDirectory`)**: เป็นโฟลเดอร์สำหรับเอกสารสำคัญของแอป **ไม่มีทางโดนระบบปฏิบัติการลบทิ้งอย่างเด็ดขาด** ยกเว้นตัวผู้ใช้งานจะเป็นคนกดถอนการติดตั้งแอปพลิเคชัน (Uninstall) ออกไปด้วยตัวเอง

ดังนั้น หากเราจำเป็นต้องทำระบบบันทึกรูปโปรไฟล์ออฟไลน์ให้อยู่ได้คงทนถาวร เราต้องคัดลอกไฟล์ย้ายออกจากห้องแคชด้วยไลบรารี **`expo-file-system`**:

```
import * as FileSystem from 'expo-file-system';

const copyToPrivateStorage = async (temporaryUri: string): Promise<string
| null> => {
  try {
    const filename = `profile_avatar_${Date.now()}.jpg`; // ตั้งชื่อใหม่ไม่ให้ชนกัน
    const destinationUri = `${FileSystem.documentDirectory}${filename}`;

  // ย้ายไฟล์จาก Cache ไปยังโฟลเดอร์ Document ส่วนตัวของแอป
    await FileSystem.copyAsync({
      from: temporaryUri,
      to: destinationUri,
```

```
    });

    console.log('ไฟล์ถูกเก็บไว้แบบถาวรเฉพาะตัวแอปที่:', destinationUri);
    return destinationUri; // คืนค่าที่อยู่ใหม่เพื่อนำไปเซฟลง AsyncStorage ต่อไป
  } catch (error) {
    console.error('คัดลอกไฟล์ Sandbox ล้มเหลว:', error);
    return null;
  }
};
```

11.6 ระบบจัดการไฟล์แบบที่ 2: Shared Media Storage (แอปอื่นมองเห็นและเข้าถึงข้อมูลร่วมกันได้)

หากแอปพลิเคชันของเราต้องการถ่ายรูปภาพ แล้วบันทึกส่งออกไปจัดแสดงอยู่ในแกลเลอรีหลักของโทรศัพท์มือถือ (Camera Roll / Photos App) เพื่อให้แอปพลิเคชันภายนอก (เช่น Facebook, Instagram, LINE) ค้นหาและหยิบภาพนั้นไปใช้งานต่อได้ เราจะไม่บันทึกใน Sandbox แต่จะต้องบันทึกเข้าสู่คลังภาพสาธารณะของระบบ โดยใช้ไลบรารี `expo-media-library`

การติดตั้ง:

```
npx expo install expo-media-library
```

การตั้งค่าการขอสิทธิ์เข้าใช้งานใน app.json (สำหรับสร้าง Asset):

```
{
 "expo": {
  "plugins": [
   [
    "expo-media-library",
        {
     "photosPermission": "แอปต้องการขอสิทธิ์เข้าถึงเพื่อบันทึกไฟล์ภาพถ่ายลงสู่แกลเลอรีของคุณ"
        }
   ]
  ]
 }
}
```

ตัวอย่างการส่งออกไฟล์ไปที่คลังแกลเลอรีสาธารณะ:

เมื่อเรียกใช้งานคำสั่ง createAssetAsync รูปจะถูกนำไปบรรจุลงในคลังภาพส่วนกลางของตัวเครื่องทันที และเปิดโอกาสให้ระบบแอปอื่น ๆ สแกนตรวจพบภาพนี้ได้:

```
import * as MediaLibrary from 'expo-media-library';
import { Alert } from 'react-native';

const saveToSharedGallery = async (localUri: string) => {
  try {
  // 1. ตรวจสอบสิทธิ์เข้าถึงคลังสื่อในเครื่องก่อนเขียนข้อมูล
    const { status } = await MediaLibrary.requestPermissionsAsync();
    if (status !== 'granted') {
      Alert.alert('สิทธิ์ถูกปฏิเสธ', 'กรุณาอนุมัติสิทธิ์เข้าถึงแกลเลอรีสาธารณะ');
      return;
    }

  // 2. สร้างสินทรัพย์ภาพ (Asset) บันทึกลงเครื่องภายนอกแซนด์บ็อกซ์
    const asset = await MediaLibrary.createAssetAsync(localUri);
    Alert.alert('บันทึกสำเร็จ!', 'รูปภาพของท่านถูกจัดเก็บลงแกลเลอรีส่วนกลางแล้ว');
    console.log('Asset ID:', asset.id);
  } catch (error) {
    console.error('เกิดข้อผิดพลาดในการบันทึกลงแกลเลอรีหลัก:', error);
  }
};
```

11.7 ระบบจัดการไฟล์แบบที่ 3: Cloud Storage Network (อัปโหลด/ดาวน์โหลดผ่าน Supabase Storage)

หากรูปโปรไฟล์นั้นต้องซิงก์แลกเปลี่ยนนำไปเปิดดูในสมาร์ทโฟนของเพื่อน หรือนำไปแสดงผลบนเว็บไซต์กลาง ข้อมูลภาพถ่ายจะไม่สามารถเก็บอยู่เพียงในเครื่องเครื่องเดียวได้ เราจึงจำเป็นต้องอัปโหลดส่งไฟล์ผ่านทางเครือข่ายเครือข่าย (Network System) ขึ้นไปเก็บไว้บนคลาวด์ เช่น บริการ **Supabase Storage Buckets**

กลไกการอัปโหลดไฟล์รูปภาพใน React Native ไปยัง Supabase:

เนื่องจากสภาพแวดล้อมระบบ React Native ไม่มีฟังก์ชันจัดการ Blob หรือ File แบบมาตรฐานเดียวกันกับบราวเซอร์คอมพิวเตอร์ วิธีการส่งไฟล์ที่เสถียรที่สุดคือการอ่านไฟล์เป็นรูปแบบสตริงรหัส **Base64** จากดิสก์ด้วย `expo-file-system` แล้วทำการแกะถอดรหัสออกมาเป็นรหัสฐานข้อมูลแบบไบนารี **`ArrayBuffer`** ก่อนส่งยิงขึ้น API:

1. การติดตั้งไลบรารีแปลงข้อมูล Base64:

```
npm install base64-js
```

2. โค้ด Asynchronous สำหรับอัปโหลดขึ้น Supabase Storage:

ผู้พัฒนาต้องเข้าไปที่แดชบอร์ดของ Supabase แล้วเปิดสร้างโฟลเดอร์ฝากไฟล์ที่เรียกว่า **Bucket** (เช่นสร้างชื่อตารางฝากไฟล์ชื่อ `avatars`) และตั้งค่านโยบายสิทธิ์ (RLS Rules) ให้เป็น Public หรืออนุญาตสิทธิ์การอัปโหลดให้พร้อมใช้งาน:

```
import { supabase } from '../lib/supabase';
import * as FileSystem from 'expo-file-system';
import { decode } from 'base64-js';

const uploadToSupabaseCloud = async (localFileUri: string): Promise<string
| null> => {
  try {
  // 1. อ่านข้อมูลไฟล์ภาพจากเครื่องปลายทางออกมาเป็น Base64 String
    const base64Data = await FileSystem.readAsStringAsync(localFileUri, {
      encoding: FileSystem.EncodingType.Base64,
    });

  // 2. แปลงสตริงรหัส Base64 ให้กลายเป็น ArrayBuffer
    const arrayBuffer = decode(base64Data);

  // 3. กำหนดชื่อและพาธปลายทางที่จะบันทึกไว้ในระบบคลาวด์
    const fileName = `profile_${Date.now()}.jpg`;
    const filePath = `user_avatar/${fileName}`;

  // 4. สั่งส่งอัปโหลด (Upload) ข้อมูลขึ้นไปฝากที่คลาวด์บัคเก็ต 'avatars'
    const { data, error } = await supabase.storage
   .from('avatars') // ระบุชื่อ Bucket เป้าหมาย
   .upload(filePath, arrayBuffer, {
        contentType: 'image/jpeg',
        upsert: true // หากเจอพาธเดียวกันให้สั่งเขียนทับทันที
      });

    if (error) {
      throw error;
    }

  // 5. ดึงข้อมูลที่อยู่พาธ URL สาธารณะ (Public URL) สำหรับนำไปใช้เรนเดอร์แสดงผล
    const { data: publicUrlData } = supabase.storage
   .from('avatars')
   .getPublicUrl(filePath);

    console.log('อัปโหลดขึ้นคลาวด์สำเร็จ! ลิงก์รูปภาพสาธารณะคือ:', publicUrlData.publicUrl);
```

```
    return publicUrlData.publicUrl; // ลิงก์ตรงที่นำไปส่งต่อให้ API อื่นใช้งานได้ทั่วโลก

  } catch (error: any) {
    console.error('ไม่สามารถส่งข้อมูลขึ้นระบบคลาวด์ได้:', error.message || error);
    return null;
  }
};
```

สรุปเปรียบเทียบระบบจัดเก็บไฟล์ทั้ง 3 รูปแบบ

ตารางที่ 11.1 สรุปเปรียบเทียบระบบจัดเก็บไฟล์ทั้ง 3 รูปแบบ

รูปแบบการจัดเก็บ	พื้นที่จัดเก็บ (Directory/Server)	การแบ่งปัน (Sharing)	ความคงทนถาวร (Persistence)
1. Private Sandbox	โฟลเดอร์เอกสารภายในแอป (documentDirectory)	เฉพาะแอปตนเองเท่านั้น แอปอื่นไม่มีสิทธิ์มาแตะต้อง	ถาวรในเครื่อง จนกว่าแอปพลิเคชันจะถูกถอนออก
2. Shared Storage	แกลเลอรีสื่อหลักของโทรศัพท์ (expo-media-library)	แบ่งปันสาธารณะในเครื่อง แอปอื่นเปิดแกลเลอรีก็สแกนค้นเจอ	ถาวรตามสิทธิ์ผู้ใช้ดำรงอยู่ตลอดไป จนกว่าจะกดยืนยันลบในคลังภาพ
3. Cloud Storage	บนเซิร์ฟเวอร์ฝากไฟล์คลาวด์ (Supabase Storage)	เข้าถึงข้ามอุปกรณ์ทั่วโลก ผ่านการกระจายตัวของ URL เครือข่าย	ถาวรตามระบบเซิร์ฟเวอร์ ลบออกเมื่อมีคำสั่ง API Trigger ลบทิ้งเท่านั้น

สรุปท้ายบทเรียน

กลไกการจัดการสิทธิ์กล้องถ่ายรูปและคลังภาพ: การเข้าถึงกล้องและภาพถ่ายในเครื่องผู้ใช้จำเป็นต้องทำตามมาตรการความปลอดภัยและได้รับอนุญาตจากระบบปฏิบัติการ โดยเราสามารถตรวจสอบและขอสิทธิ์ใช้งานเลนส์กล้องแยกด้วย useCameraPermissions Hook ของ expo-camera และขอสิทธิ์อ่านคลังรูปภาพแกลเลอรีด้วยฟังก์ชันขออนุญาตของ ImagePicker (Media Library Permissions) ก่อนที่จะเรียกเข้าใช้งาน

การใช้งาน CameraView เพื่อพรีวิวภาพสด: คอมโพเนนต์ CameraView เข้ามาทำหน้าที่เปิดช่องรับสัญญาณภาพและแสดงพรีวิวภาพเคลื่อนไหวสดจากกล้องบนหน้าจอมือถือใน Expo SDK 54 โดยเราสามารถส่งผ่านพร็อพส์ facing บ่งชี้เลนส์หน้าหรือหลัง และใช้ autofocus ในการสั่งจัดระเบียบความคมชัดอัตโนมัติ ซึ่งต้องเตรียมการดีไซน์ปุ่มสัมผัสในการเปิดปิดมุมกล้องให้ใช้งานได้อย่างสะดวกสบาย

การลั่นชัตเตอร์ถ่ายภาพนิ่ง: การถ่ายภาพนิ่งแบบ Snapshot จากตัวรับพรีวิวสดจะกระทำได้โดยการประกาศผูกตัวแปรอ้างอิง Ref เข้าหาคอมโพเนนต์ CameraView และเรียกฟังก์ชัน Asynchronous ยอดนิยมอย่าง takePictureAsync ซึ่งจะบันทึกรูปภาพนิ่งชั่วคราวลงในหน่วยความจำแคชของอุปกรณ์ และส่งคืนค่าพารามิเตอร์ของรูปภาพและที่อยู่ไฟล์ (URI) กลับมาประมวลผลต่อ

การเลือกรูปภาพและสื่อผ่าน expo-image-picker: เพื่ออำนวยความสะดวกให้ผู้ใช้งานสามารถดึงข้อมูลรูปภาพหรือสื่อที่มีการเก็บบันทึกไว้เดิมในเครื่องมาประยุกต์ใช้งาน (เช่น การเลือกภาพอวาตาร์) เราจะประยุกต์ใช้คำสั่ง launchImageLibraryAsync ของ expo-image-picker ในการเปิดหน้าต่างแกลเลอรีเนทีฟของโทรศัพท์เพื่อให้ผู้ใช้จัดแจงครอปภาพและส่งค่า URI รูปที่ถูกเลือกกลับมา

ระบบจัดการไฟล์แบบที่ 1 (Private Sandbox Storage): ไฟล์ที่ได้จากการถ่ายภาพหรือการดึงภาพจากแกลเลอรีจะได้รับการสร้างเป็นที่อยู่พาธชั่วคราวอยู่ใน cacheDirectory เสมอ ซึ่งเสี่ยงต่อการโดนระบบปฏิบัติการล้างข้อมูลทิ้งเมื่อพื้นที่เต็ม ผู้พัฒนาจึงต้องใช้ expo-file-system ในการคัดลอกไฟล์ย้ายเข้าไปจัดเก็บถาวรใน Sandbox ส่วนตัวของแอป หรือเรียกว่า documentDirectory ซึ่งมีความปลอดภัยและระบบจะไม่เข้ามากวนข้อมูล

ระบบจัดการไฟล์แบบที่ 2 (Shared Media Storage): กรณีที่รูปถ่ายจากแอปพลิเคชันของเราต้องการส่งต่อไปใช้งานร่วมกับแกลเลอรีหลักของโทรศัพท์ เพื่อเปิดโอกาสให้แอปภายนอก เช่น LINE หรือ Facebook สแกนดึงรูปนี้ไปใช้ได้ต่อ เราจะเปลี่ยนจากการเก็บข้อมูลในแซนด์บ็อกซ์ส่วนตัวมาส่งออกข้อมูลผ่านคำสั่ง createAssetAsync ของ expo-media-library เพื่อไปจดทะเบียนลงทะเบียนไฟล์ในคลังภาพสาธารณะของระบบ

ระบบจัดการไฟล์แบบที่ 3 (Cloud Storage Network): เพื่อให้รูปภาพถ่ายสามารถเผยแพร่ข้ามอุปกรณ์และส่งต่อขึ้นฝั่งเว็บไซต์ได้ ข้อมูลรูปภาพจำเป็นต้องอัปโหลดขึ้นฝั่งคลาวด์ เช่น Supabase Storage Buckets โดย React Native จะต้องใช้ expo-file-system ในการอ่านข้อมูลจากพิกัดไฟล์ออกมาเป็นสตริงรหัส Base64 จากนั้นใช้ไลบรารี base64-js ถอดรหัสเป็น ArrayBuffer และส่งยิงอัปโหลดขึ้นเซิร์ฟเวอร์เพื่อให้ได้ลิงก์ตรง (Public URL) สาธารณะ

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความรู้และความเข้าใจเกี่ยวกับแนวคิดระบบความปลอดภัยของภาพถ่ายเนทีฟ, พฤติกรรมการทำงานของคอมโพเนนต์ `CameraView` และ `expo-image-picker` ใน Expo SDK 54, ตลอดจนกลไกสถาปัตยกรรมการจัดเก็บรูปภาพ 3 รูปแบบหลัก ได้แก่ ระบบจัดเก็บไฟล์เฉพาะแอป (Sandbox), การส่งออกไฟล์สาธารณะข้ามแอป (Shared Gallery) และระบบจัดเก็บไฟล์ข้ามเครือข่ายบนคลาวด์ (Supabase Storage)

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: ข้อใดอธิบายพฤติกรรมการเข้าถึงและขออนุมัติใช้งานสื่อรูปภาพแบบเนทีฟได้ถูกต้องที่สุด?

ก. เมื่อแอปพลิเคชันได้รับสิทธิ์เปิดใช้กล้องถ่ายรูปแล้ว จะสามารถดึงประวัติรูปภาพเก่าในคลังอัลบั้มมาประมวลผลได้ทันทีโดยไม่ต้องร้องขอสิทธิ์แกลเลอรี

ข. เนื่องจากเกี่ยวข้องกับสิทธิ์ความเป็นส่วนตัวต่างประเภทกัน ระบบจึงบังคับแยกขอสิทธิ์ระหว่างกล้องถ่ายรูป (Camera Permission) และคลังรูปภาพในอุปกรณ์ (Media Library / Photos Permission) แยกจากกันเป็นเอกเทศ

ค. สิทธิ์ในการเข้าถึงคลังสื่อภาพนิ่งจะถูกสั่งอนุมัติให้อัตโนมัติในฝั่งเซิร์ฟเวอร์โดยที่ผู้ใช้งานมือถือไม่มีสิทธิ์ปฏิเสธ

ง. สิทธิ์การใช้กล้องของระบบเนทีฟทำงานโดยไม่ต้องอาศัยการกำหนดค่าใด ๆ ในไฟล์ `app.json`

ข้อ 2: คอมโพเนนต์ใดที่ได้มาตรฐานของ Expo SDK 54 ที่แนะนำให้นักพัฒนาใช้สร้างหน้าต่างเลนส์พรีวิวภาพสด (Camera Live Preview) จากเลนส์มือถือ?

ก. `<Camera>`

ข. `<CameraPreview>`

ค. `<CameraView>`

ง. `<NativeLens>`

ข้อ 3: การตั้งค่าสลับการใช้หน้าเลนส์พรีวิวสดของกล้องถ่ายรูปสลับไปมาระหว่าง "กล้องหน้า" และ "กล้องหลัง" บนคอมโพเนนต์กล้องรุ่นใหม่ ต้องควบคุมค่าที่พร็อพส์ (Prop) และส่งค่าใด?

ก. พร็อพส์ `type` รับค่า `front` หรือ `back`

ข. พร็อพส์ `facing` รับค่า `front` หรือ `back`

ค. พร็อพส์ `lens` รับค่า `selfie` หรือ `rear`

ง. พร็อพส์ `facing` รับค่า `user` หรือ `environment`

ข้อ 4: พฤติกรรมการกักเก็บรูปภาพเริ่มต้นของผลลัพธ์ที่ได้จากการยิงคำสั่งถ่ายภาพ `takePictureAsync()` หรือเลือกภาพจาก `launchImageLibraryAsync()` ข้อใดกล่าวได้ถูกต้อง?

ก. ข้อมูลไบนารีของภาพถ่ายทั้งหมดจะถูกเขียนพ่วงบันทึกลงฐานข้อมูลหลักของเครื่องโดยทันที

ข. ผลลัพธ์จะถูกจัดเก็บเป็นที่อยู่พาธภาพถ่ายชั่วคราวชี้ไปที่โฟลเดอร์แคช (`cacheDirectory`) ซึ่งระบบปฏิบัติการ (OS) มีสิทธิ์สุ่มล้างข้อมูลทิ้งได้ตลอดเวลาเมื่อดิสก์เครื่องเต็ม

ค. ตัวไฟล์ภาพจะถูกกักเก็บไว้อยู่ในสารระบบ `documentDirectory` แบบคงทนถาวรทันทีหลังเสร็จสิ้นกระบวนการ

ง. ไฟล์ภาพจะถูกส่งอัปโหลดผ่านระบบเครือข่ายอินเทอร์เน็ตขึ้นคลาวด์โดยทันทีโดยไม่ต้องรอเขียนโปรแกรมเชื่อมต่อ

ข้อ 5: แฟ้มไดเรกทอรีระบบใดภายใน Sandbox ของแอปพลิเคชันที่มีคุณสมบัติเก็บรักษาทรัพยากรได้อย่างถาวรปลอดภัย ไม่มีวันโดนระบบปฏิบัติการลบทิ้ง ยกเว้นผู้ใช้จะสั่งถอนการติดตั้งแอป?

ก. `FileSystem.cacheDirectory`

ข. `FileSystem.temporaryDirectory`

ค. `FileSystem.documentDirectory`

ง. `FileSystem.sharedDirectory`

ข้อ 6: หากต้องการพัฒนาแอปพลิเคชันถ่ายรูปแล้วสั่งบันทึกส่งออกไฟล์ภาพไปเก็บไว้ในอัลบั้มหลักของมือถือ เพื่อให้แอปพลิเคชันอื่นนอกตัวแอป (เช่น Facebook, IG) สามารถดึงรูปนี้ไปใช้ได้ด้วย จะต้องบูรณาการผ่านคำสั่งของไลบรารีใด?

ก. `expo-file-system`

ข. `expo-media-library`

ค. `expo-image-picker`

ง. `expo-sharing`

ข้อ 7: คำสั่งใดของไลบรารี `expo-media-library` ที่ใช้ระบุตัวเลือกสั่งบันทึกไฟล์ภาพเนทีฟตรงเข้าสู่แกลเลอรีแชร์ข้ามแอปพลิเคชัน?

ก. `MediaLibrary.saveToLibraryAsync()` หรือ `MediaLibrary.createAssetAsync()`

ข. `MediaLibrary.uploadImageAsync()`

ค. `MediaLibrary.copyToSharedSpace()`

ง. `MediaLibrary.exportPhoto()`

ข้อ 8: การอัปโหลดไฟล์รูปภาพจากฝั่ง React Native ขึ้นสู่ระบบฝากไฟล์ Supabase Storage Buckets วิธีการส่งชุดข้อมูลวิธีใดที่ทำงานได้เสถียรที่สุดเพื่อหลีกเลี่ยงข้อจำกัดการอ่านไฟล์แบบ Blob เนทีฟ?

ก. การดึงที่อยู่พาธ URI ไปยิงคำสั่งอัปโหลดผ่าน REST API ตรง ๆ โดยไม่ผ่าน SDK

ข. การอ่านไฟล์ข้อมูลออกมาเป็นรหัสสตริง Base64 แล้วทำการแปลงถอดรหัสออกมาเป็นแบบ `ArrayBuffer` ไบนารี

ค. การดึงพิกเซลสีทั้งหมดมาเรนเดอร์ในคอมโพเนนต์ Canvas แล้วเก็บรหัส JSON

ง. การอัปโหลดเฉพาะชื่อไฟล์ขึ้นคลาวด์ แล้วให้ฝั่งคลาวด์ดาวน์โหลดไฟล์จากมือถือกลับไปเอง

ข้อ 9: ในกระบวนการอ่านไฟล์และแปลงรหัส Base64 จากพื้นที่จัดเก็บเครื่องฝั่ง Client ของโทรศัพท์มือถือเป็น Asynchronous จะต้องสั่งการเรียกใช้โมดูลใดเป็นแกนหลัก?

ก. `FileSystem.readAsStringAsync()` ร่วมกับออปชัน `{ encoding: FileSystem.EncodingType.Base64 }`

ข. `ImagePicker.readAsBase64Async()`

ค. `AsyncStorage.getItem()` ร่วมกับคีย์ไฟล์

ง. `MediaLibrary.readBytesAsync()`

ข้อ 10: หลังจากที่ทำการส่งอัปโหลดไฟล์เข้าไปใน Supabase Storage Bucket เป็นที่เรียบร้อย หากต้องการลิงก์ตรงภายนอก (Direct URL) สำหรับนำมาเรนเดอร์แสดงผลในแอปทันที จะต้องเรียกใช้เมธอดสืบค้นใดของ Supabase Client?

ก. `supabase.storage.from('bucket').getPublicUrl(filePath)`

ข. `supabase.storage.from('bucket').download(filePath)`

ค. `supabase.storage.from('bucket').select('url')`

ง. `supabase.storage.from('bucket').retrieveLink(filePath)`

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** การอนุญาตสิทธิ์กล้อง ไม่ได้ครอบคลุมการสืบค้นภาพเก่าในคลังอัลบั้ม ผู้ใช้ต้องเลือกยินยอมสิทธิ์แยกส่วนกัน

• **ข. ถูก:** ระบบความปลอดภัยของ iOS/Android แยกสิทธิ์การใช้เลนส์สัมผัสกล้อง (Camera) และการอ่านคลังไฟล์ส่วนตัวในอัลบั้มเครื่อง (Photos / Media Library) ออกจากกันอย่างเด็ดขาดเพื่อความเป็นส่วนตัวสูงสุดของผู้ใช้

• **ค. และ ง. ผิด:** ระบบสิทธิ์ถูกควบคุมโดย OS ฝั่งมือถือ ไม่ใช่ฝั่งเซิร์ฟเวอร์ และจำเป็นต้องมีการลงทะเบียนไว้ในไฟล์ `app.json` เสมอ

เฉลย ข้อ 2: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ข. และ ง. ผิด:** ไม่มีส่วนคอมโพเนนต์ชื่อเหล่านี้จัดเตรียมไว้ใน API ของ Expo SDK 54

• **ค. ถูก:** Expo ได้ยุบและปรับปรุงคอมโพเนนต์การพรีวิวเลนส์กล้องตัวเก่า (`Camera`) และเปิดตัวคอมโพเนนต์ `` `CameraView` `` ขึ้นมาขับเคลื่อนแทนในแพ็คเกจ `expo-camera`

เฉลย ข้อ 3: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** พร็อพส์ `type` ถูกเปลี่ยนชื่อออกไปแล้วใน API รุ่นใหม่

• **ข. ถูก:** พร็อพส์ที่ระบุหน้าเลนส์คือ `` `facing` `` โดยยอมรับค่าสตริงสองชนิดคือ `'front'` (เซลฟี่) หรือ `'back'` (กล้องหลัง) เท่านั้น

• **ค. และ ง. ผิด:** ค่าโหมดหรือเลนส์เหล่านี้ไม่เข้าเกณฑ์พารามิเตอร์ของ Expo Camera API

เฉลย ข้อ 4: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** จะไม่มีการเขียนหรือสร้างข้อมูลลงในฐานข้อมูลหลักให้อัตโนมัติ

• **ข. ถูก:** ทั้งภาพที่ถ่ายสดหรือดึงจาก Picker จะได้รับการสร้างไว้ในห้องแคชชั่วคราว (`cacheDirectory`) ซึ่งมีความเสี่ยงสูงที่จะโดนกวาดล้างลบไฟล์ทิ้งโดยระบบ OS เมื่อดิสก์มีความจุน้อยลงหรือแอปพลิเคชันปิดการทำงาน

• **ค. และ ง. ผิด:** ไม่ได้เข้าสารระบบ Document อัตโนมัติ และไม่สามารถวิ่งส่งขึ้นระบบคลาวด์ได้เองโดยไม่มีอินเทอร์เฟซคำสั่งอัปโหลด

เฉลย ข้อ 5: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. และ ข. ผิด:** โฟลเดอร์ห้อง Cache หรือ Temporary มีสิทธิ์โดนลบขยะได้ตลอดเวลาโดย OS

• **ค. ถูก:** แฟ้มเอกสาร `**FileSystem.documentDirectory**` คือพื้นที่กักเก็บไฟล์ส่วนตัวแบบปลอดภัยภายใน Sandbox ของตัวแอปพลิเคชัน ซึ่งปลอดภัยจากการลบกวาดล้างของ OS เสมอ

• **ง. ผิด:** ไม่มีไดเรกทอรีชื่อนี้

เฉลย ข้อ 6: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `expo-file-system` ใช้จัดเก็บไฟล์ภายในแซนด์บ็อกซ์ส่วนตัวของตัวแอปเองเป็นหลัก

• **ข. ถูก:** `**expo-media-library**` คือเครื่องมือและไลบรารีที่เปิดทางให้แอปพลิเคชันเข้าถึง จัดการ และเซฟสื่อบันทึกลงสู่โฟลเดอร์ภาพสาธารณะ (Shared Storage) เพื่อแชร์แชร์ข้ามแอปพลิเคชันเครื่องได้สำเร็จ

• **ค. และ ง. ผิด:** `expo-image-picker` ใช้เปิดเลือกรูป ไม่ใช่การเซฟบันทึกคลังภาพ ส่วน `expo-sharing` ใช้ยิงหน้าต่างระบบแชร์ไฟล์ระบบ (Share Sheet) ไม่ได้จัดเก็บถาวรในแกลเลอรี

เฉลย ข้อ 7: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก:** ฟังก์ชันที่รับหน้าที่จัดสร้าง asset รูปภาพและนำเข้าคลังรูปสาธารณะคือ `MediaLibrary.createAssetAsync()` หรือสั่งบันทึกไฟล์ภาพเนทีฟตรงด้วย `MediaLibrary.saveToLibraryAsync()`

• **ข. ค. และ ง. ผิด:** ชื่อเมธอดไม่มีระบุไว้ในสารระบบของคลังคำสั่ง `expo-media-library`

เฉลย ข้อ 8: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ค. และ ง. ผิด:** การทำ Blob ใน React Native เจอปัญหาความเข้ากันได้ และการส่งพิกเซลสีหรือส่งเพียงชื่อไฟล์ไม่สามารถย้ายไฟล์จริงขึ้นคลาวด์ได้

• **ข. ถูก:** การสกัดดึงข้อมูลออกมาเป็นสตริงความปลอดภัยรหัส Base64 ด้วย `expo-file-system` แล้วใช้ฟังก์ชันถอดค่าออกมาเป็นไบนารีแบบ `ArrayBuffer` ถือเป็นรูปแบบมาตรฐานที่เสถียรที่สุดในการยิงรีเควสผ่าน `@supabase/supabase-js`

เฉลย ข้อ 9: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก:** โมดูล Asynchronous ของระบบไฟล์ของ Expo คือ `FileSystem.readAsStringAsync` ซึ่งเราต้องกำหนดคีย์ออปชันรหัสผ่านเป็น `{ encoding: FileSystem.EncodingType.Base64 }`

• **ข. ค. และ ง. ผิด:** คำสั่งอื่นไม่มีอยู่จริง หรือใช้เก็บบันทึกความสัมพันธ์ตัวเลขธรรมดาอย่าง AsyncStorage ไม่สามารถแปลงไฟล์ไบนารีได้

เฉลย ข้อ 10: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก:** เมธอดสืบค้นลิงก์ตรงคือ `.getPublicUrl(filePath)` ซึ่งจะส่งอ็อบเจกต์ที่มีแอตทริบิวต์ระบุพาธ `.data.publicUrl` กลับออกมาให้ใช้ส่งเข้า Image Component เพื่อเรนเดอร์ภาพจากอินเทอร์เน็ตได้ทันที

• **ข. ผิด:** `.download()` คุมการรับข้อมูลไบนารีดิบลงหน่วยความจำ ไม่ใช่ขอลิงก์ URL

• **ค. และ ง. ผิด:** โครงสร้างคำสั่งเหล่านี้ระบุพารามิเตอร์และ API ชื่อไม่ถูกต้อง

ปฏิบัติการท้ายบทเรียน: การพัฒนาแอปเลือกรูปโปรไฟล์อวาตาร์บันทึก 3 ระบบ

ปฏิบัติการนี้จะฝึกทักษะระดับสูงในการทำโมดูลอวาตาร์ โดยเชื่อมประสานกล้องพรีวิวสดของระบบเนทีฟและการกักเก็บไฟล์ 3 ระบบ ได้แก่ **Private Sandbox** (ในดิสก์จำกัดเฉพาะแอป), **Shared Media Library** (ในดิสก์สาธารณะแชร์ข้ามแอป) และ **Cloud Network Storage** (อัปโหลดเซิร์ฟเวอร์ Supabase Storage)

1. วัตถุประสงค์การเรียนรู้

1. สามารถเขียนขอยื่นสิทธิ์ความปลอดภัยเข้าใช้กล้องถ่ายรูป อัลบั้มส่วนตัว และการเขียนสื่อลงแกลเลอรีสาธารณะได้สำเร็จ

2. สามารถควบคุมกล้องเซลฟี่สดผ่าน **CameraView** และสลับกล้องหน้าหลังรวมถึงสั่งชัตเตอร์ Asynchronous

3. สามารถเรียกใช้ **expo-file-system** ย้ายไฟล์ภาพจาก Cache Folder ไปฝากถาวรใน **documentDirectory** ภายใน Sandbox

4. สามารถประยุกต์ใช้งาน **expo-media-library** ส่งออกไฟล์ภาพไปเก็บบันทึกในคลังภาพสาธารณะของระบบเครื่องเพื่อแชร์ข้ามแอปพลิเคชัน

5. สามารถเขียนตรรกะการแปลงสตริง Base64 และ ArrayBuffer เพื่อยิงภาพอัปโหลดตรงขึ้นระบบฝากไฟล์ Supabase Storage Bucket และดึง URL ลิงก์ตรงมาจัดแสดงผล

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนต้องจัดสร้างโปรเจกต์ React Native Expo โดยมีโครงสร้างดังนี้:

```
โครงสร้างไฟล์ปฏิบัติการ:
├── .env                                  => บรรจุคีย์โครงการ EXPO_PUBLIC_SUPABASE_URL และ EXPO_PUBLIC_SUPABASE_ANON_KEY
├── app.json                              => ลงทะเบียนคำอนุญาตสิทธิ์ใช้งานกล้อง อัลบั้มภาพ และคลังภาพส่วนกลาง
├── lib/
│   └── supabase.ts                       => สร้างอินสแตนซ์ Supabase Client
└── app/
    └── index.tsx                         => หน้าจอจัดแสดงโปรไฟล์ แผงกล้อง Modal และฟังก์ชันการเขียนไฟล์ 3 ระบบ
```

การตั้งค่าตารางจัดเก็บไฟล์ฝากคลาวด์บน Supabase (Supabase Storage Setup):

1. ให้ผู้เรียนไปที่คอนโซลของ Supabase เข้าหัวข้อ **Storage**

2. กดสร้าง Bucket ใหม่ขึ้นมาชื่อว่า **`avatars`**

3. ปรับเปลี่ยนนโยบายความปลอดภัย (RLS Policies) ของบัคเก็ต **avatars** นี้โดยกดเพิ่มกฎใหม่ (New Policy) -> เลือกเทมเพลต **"Enable read access to everyone"** (เพื่อให้บุคคลภายนอกสามารถดึงภาพจาก URL สาธารณะได้) และกฎ **"Enable insert/upload access for authenticated and anonymous users"** (สำหรับการเปิดให้อัปโหลดไฟล์ส่งขึ้นไป)

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: ติดตั้งคลังไลบรารีทั้งหมด

เปิดใช้งานเทอร์มินัลในโฟลเดอร์โปรเจกต์ของคุณแล้วสั่งรันบรรทัดต่อไปนี้:

```
npx expo install expo-camera expo-image-picker expo-file-system expo-
media-library @react-native-async-storage/async-storage base64-js
```

ขั้นตอนที่ 2: ตั้งค่าระบบขอสิทธิ์ในไฟล์ app.json

เปิดไฟล์ app.json แล้วระบุ Plugins สิทธิ์เข้าใช้งานของอุปกรณ์ให้เรียบร้อย:

```
{
 "expo": {
  "name": "MultiAvatarApp",
  "slug": "multi-avatar-app",
  "version": "1.0.0",
  "plugins": [
   [
    "expo-camera",
        {
     "cameraPermission": "แอปต้องการขอสิทธิ์เข้าใช้งานกล้องเพื่อใช้ในการถ่ายภาพโปรไฟล์"
      }
   ],
   [
    "expo-image-picker",
        {
     "photosPermission": "แอปต้องการขอสิทธิ์เข้าถึงคลังสื่อภาพเพื่อดึงภาพมาอัปเกรดโปรไฟล์"
        }
   ],
   [
    "expo-media-library",
        {
     "photosPermission": "แอปต้องการขอสิทธิ์เขียนบันทึกไฟล์ภาพถ่ายของแอปเพื่อนำไปบันทึกลงคลังแกลเลอรี
เครื่อง"
        }
   ]
  ]
 }
}
```

ขั้นตอนที่ 3: จัดการระบุคีย์ลับเซิร์ฟเวอร์ใน lib/supabase.ts

สร้างไฟล์ .env ไว้ที่โฟลเดอร์ชั้นนอกสุดของโปรเจกต์เพื่อนำส่งคีย์ลับ:

```
EXPO_PUBLIC_SUPABASE_URL=https://your-project-id.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=your-anon-public-key-here
```

และสร้างไฟล์เชื่อมโยง lib/supabase.ts:

```
import 'react-native-url-polyfill/auto';
import { createClient } from '@supabase/supabase-js';

const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL || '';
const supabaseAnonKey = process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY || '';

export const supabase = createClient(supabaseUrl, supabaseAnonKey);
```

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

ผู้เรียนสามารถนำโค้ดสำหรับควบคุม จัดการรูปโปรไฟล์และประมวลผลเซฟและอัปโหลดส่งไฟล์ 3 ระบบนี้ บันทึกลงในไฟล์ `app/index.tsx` เพื่อรันระบบทดสอบระดับเนทีฟ:

```
import React, { useState, useEffect, useRef } from 'react';
import {
  StyleSheet,
  Text,
  View,
  Image,
  TouchableOpacity,
  Modal,
  ActivityIndicator,
  Alert,
  SafeAreaView,
  ScrollView
} from 'react-native';
import { CameraView, CameraType, useCameraPermissions } from 'expo-
camera';
import * as ImagePicker from 'expo-image-picker';
import * as FileSystem from 'expo-file-system';
import * as MediaLibrary from 'expo-media-library';
import AsyncStorage from '@react-native-async-storage/async-storage';
import { decode } from 'base64-js';
import { supabase } from '../lib/supabase';

const LOCAL_STORAGE_KEY = '@local_avatar_path';
const CLOUD_STORAGE_KEY = '@cloud_avatar_url';

export default function App() {
  const [tempImageUri, setTempImageUri] = useState<string | null>(null);

 // สถานะพาธและผลลัพธ์จัดเก็บรูปภาพในระบบต่าง ๆ
  const [sandboxPath, setSandboxPath] = useState<string | null>(null);
  const [sharedGalleryStatus, setSharedGalleryStatus] = useState<string |
```

```
null>(null);
 const [cloudUrl, setCloudUrl] = useState<string | null>(null);

  const [isCameraVisible, setIsCameraVisible] = useState(false);
  const [facing, setFacing] = useState<CameraType>('front');
  const [isProcessing, setIsProcessing] = useState(false);

  const [cameraPermission, requestCameraPermission] =
useCameraPermissions();
  const [mediaLibraryPermission, requestMediaLibraryPermission] =
MediaLibrary.usePermissions();
  const cameraRef = useRef<CameraView | null>(null);

 // 1. โหลดข้อมูลที่เคยเก็บจำไว้ตอนเปิดเข้าแอป
  useEffect(() => {
    loadSavedSettings();
  }, []);

  const loadSavedSettings = async () => {
    try {
      const savedLocal = await AsyncStorage.getItem(LOCAL_STORAGE_KEY);
      const savedCloud = await AsyncStorage.getItem(CLOUD_STORAGE_KEY);

      if (savedLocal) {
        const info = await FileSystem.getInfoAsync(savedLocal);
        if (info.exists) {
          setSandboxPath(savedLocal);
        }
      }
      if (savedCloud) {
        setCloudUrl(savedCloud);
   }
    } catch (e) {
      console.error(e);
    }
  };

 // 2. ดึงภาพถ่ายจากอัลบั้มเครื่อง
  const handlePickImage = async () => {
    const { status } = await
ImagePicker.requestMediaLibraryPermissionsAsync();
    if (status !== 'granted') {
      Alert.alert('คำเตือน', 'กรุณาเปิดสิทธิ์เข้าใช้คลังภาพในการตั้งค่าเครื่อง');
      return;
    }

    const result = await ImagePicker.launchImageLibraryAsync({
      mediaTypes: ImagePicker.MediaTypeOptions.Images,
      allowsEditing: true,
      aspect: [1, 1],
```

```
      quality: 0.8
    });

    if (!result.canceled && result.assets && result.assets.length > 0) {
      setTempImageUri(result.assets[0].uri);
   // เคลียร์ผลลัพธ์การเซฟรอบเก่า
      setSandboxPath(null);
      setSharedGalleryStatus(null);
    setCloudUrl(null);
    }
  };

// 3. เริ่มเปิดใช้งานเลนส์กล้องพรีวิวสด
  const handleStartCamera = async () => {
    if (!cameraPermission || !cameraPermission.granted) {
      const response = await requestCameraPermission();
      if (!response.granted) {
        Alert.alert('สิทธิ์ถูกปฏิเสธ', 'กรุณายอมรับสิทธิ์ใช้งานเลนส์ถ่ายรูป');
        return;
      }
    }
    setIsCameraVisible(true);
  };

  const handleCapturePhoto = async () => {
    if (cameraRef.current) {
      try {
        const photo = await cameraRef.current.takePictureAsync({
          quality: 0.8,
          skipProcessing: false
        });
        if (photo && photo.uri) {
          setTempImageUri(photo.uri);
          setIsCameraVisible(false);
          setSandboxPath(null);
          setSharedGalleryStatus(null);
          setCloudUrl(null);
        }
      } catch (e) {
        Alert.alert('ผิดพลาด', 'ไม่สามารถบันทึกรูปภาพจากตัวเลนส์ได้');
      }
    }
  };

// 4. บูรณาการระบบบันทึกรูปโปรไฟล์ 3 ระดับพร้อมกัน (Multi-Destination Process)
  const handleSaveAndSyncAll = async () => {
    if (!tempImageUri) {
      Alert.alert('แจ้งเตือน', 'กรุณาถ่ายรูปหรือเลือกภาพจากแกลเลอรีก่อนกดบันทึก');
      return;
```

```
    }

    try {
      setIsProcessing(true);

    // --- ระบบที่ 1: บันทึกเฉพาะตัวใน Private Sandbox (Document Directory) ---
      const filename = `avatar_${Date.now()}.jpg`;
      const destinationSandboxUri =
`${FileSystem.documentDirectory}${filename}`;
      await FileSystem.copyAsync({
        from: tempImageUri,
       to: destinationSandboxUri
      });
      setSandboxPath(destinationSandboxUri);
      await AsyncStorage.setItem(LOCAL_STORAGE_KEY,
destinationSandboxUri);

    // --- ระบบที่ 2: บันทึกลง Shared Media Gallery (แชร์ข้ามแอปในเครื่อง) ---
      let sharedAssetId = 'ปฏิเสธสิทธิ์เข้าใช้คลังสื่อ';
      if (!mediaLibraryPermission || !mediaLibraryPermission.granted) {
        const mediaPerm = await requestMediaLibraryPermission();
        if (mediaPerm.granted) {
          const asset = await MediaLibrary.createAssetAsync(tempImageUri);
          sharedAssetId = `สำเร็จ (Asset ID: ${asset.id})`;
        }
      } else {
        const asset = await MediaLibrary.createAssetAsync(tempImageUri);
        sharedAssetId = `สำเร็จ (Asset ID: ${asset.id})`;
     }
      setSharedGalleryStatus(sharedAssetId);

    // --- ระบบที่ 3: อัปโหลดขึ้น Cloud Storage ของ Supabase (ซิงก์เน็ตเวิร์ก) ---
    // อ่านข้อมูลไฟล์เป็น Base64
      const base64Data = await FileSystem.readAsStringAsync(tempImageUri, {
     encoding: FileSystem.EncodingType.Base64
      });
    // แปลง Base64 เป็น ArrayBuffer
      const arrayBuffer = decode(base64Data);

      const cloudFilePath = `profile_pics/${filename}`;
      const { data, error } = await supabase.storage
    .from('avatars')
    .upload(cloudFilePath, arrayBuffer, {
          contentType: 'image/jpeg',
          upsert: true
        });

      if (error) {
```

```
          throw error;
        }

    // ดึงลิงก์รูปภาพสาธารณะจาก Supabase
        const { data: publicUrlData } = supabase.storage
      .from('avatars')
      .getPublicUrl(cloudFilePath);

        const publicLink = publicUrlData.publicUrl;
        setCloudUrl(publicLink);
        await AsyncStorage.setItem(CLOUD_STORAGE_KEY, publicLink);

        Alert.alert('บันทึก 3 ระบบเสร็จสิ้น!', 'ภาพถูกจัดเก็บลงดิสก์ส่วนตัว แกลเลอรีเครื่อง และอัปโหลดขึ้นคลาวด์เรียบร้อย');
      } catch (err: any) {
        Alert.alert('เกิดข้อผิดพลาดในการรันระบบบันทึก', err.message || err);
        console.error(err);
      } finally {
        setIsProcessing(false);
      }
    };

  // 5. ล้างประวัติลบรูปภาพทั้งหมดออก
    const handleClearEverything = async () => {
      Alert.alert(
        'ยืนยันการเคลียร์ข้อมูล',
        'ล้างประวัติที่อยู่และลบไฟล์ภาพในแซนด์บ็อกซ์ดิสก์ทั้งหมดหรือไม่?',
        [
          { text: 'ยกเลิก', style: 'cancel' },
          {
            text: 'ล้างข้อมูล',
            style: 'destructive',
            onPress: async () => {
              try {
        // ลบรูปภาพใน Sandbox เครื่อง
                if (sandboxPath) {
              await FileSystem.deleteAsync(sandboxPath, { idempotent: true });
                }
                await AsyncStorage.removeItem(LOCAL_STORAGE_KEY);
                await AsyncStorage.removeItem(CLOUD_STORAGE_KEY);

                setTempImageUri(null);
               setSandboxPath(null);
                setSharedGalleryStatus(null);
                setCloudUrl(null);
                Alert.alert('ล้างค่าเรียบร้อยแล้ว');
              } catch (e) {
                console.error(e);
```

```
            }
          }
        }
    ]
  );
  };

  return (
    <SafeAreaView style={styles.safeArea}>
      <ScrollView contentContainerStyle={styles.scrollContainer}>

        {/* หัวเรื่องแอป */}
        <View style={styles.header}>
          <Text style={styles.headerTitle}>📸 Profile Avatar Pro</Text>
          <Text style={styles.headerSubtitle}>ทบทวนการจัดการไฟล์ภาพถ่ายเนทีฟและคลาวด์ 3 ระบบ</Text>
        </View>

        {/* แผงเปรียบเทียบรูปภาพพรีวิว */}
        <View style={styles.avatarCard}>
          <Text style={styles.cardLabel}>อวาตาร์โปรไฟล์ปัจจุบัน</Text>
          <View style={styles.avatarWrapper}>
            {/* ดึงภาพแสดงตามลำดับความคงทน: 1. ลิงก์ตรงคลาวด์ 2. Sandbox 3. ภาพชั่วคราวดึงสด 4. ภาพว่างเปล่า */}
            {cloudUrl ? (
              <Image source={{ uri: cloudUrl }} style={styles.avatarImage} />
      ) : sandboxPath ? (
              <Image source={{ uri: sandboxPath }} style={styles.avatarImage} />
      ) : tempImageUri ? (
              <Image source={{ uri: tempImageUri }} style={styles.avatarImage} />
      ) : (
              <View style={styles.avatarPlaceholder}>
                <Text style={styles.placeholderText}>No Pic</Text>
              </View>
      )}
          </View>
          <Text style={styles.imageSourceIndicator}>
            {cloudUrl ? '☁️ แสดงจาก Cloud Storage' : sandboxPath ? '💾 แสดงจาก Local Sandbox' : tempImageUri ? '⏳ ภาพรอการบันทึก' : '⚙️ พร้อมรับข้อมูลภาพ'}
          </Text>
        </View>

        {/* แผงคำสั่งทริกเกอร์เลือกภาพ */}
        <View style={styles.buttonGroupCard}>
```

```
          <Text style={styles.cardHeaderTitle}>1. นำเข้าสื่อถ่ายภาพโปรไฟล์</Text>
          <View style={styles.rowButtons}>
            <TouchableOpacity style={styles.primaryButton}
onPress={handleStartCamera}>
              <Text style={styles.buttonText}>📷 เปิดกล้องถ่ายภาพ</Text>
            </TouchableOpacity>
            <TouchableOpacity style={[styles.primaryButton,
styles.darkBlueButton]} onPress={handlePickImage}>
              <Text style={styles.buttonText}>🖼️ ดึงจากอัลบั้มเครื่อง</Text>
            </TouchableOpacity>
          </View>
        </View>

        {/* แผงประมวลผลบันทึก 3 รูปแบบ */}
        <View style={styles.actionCard}>
          <Text style={styles.cardHeaderTitle}>2. ประมวลผลและทดสอบจัดเก็บไฟล์ 3 ทิศทาง
</Text>

          {isProcessing ? (
            <View style={styles.spinnerBlock}>
              <ActivityIndicator size="small" color="#38bdf8" />
              <Text style={styles.processingText}>กำลังย้าย Sandbox / คัดลงอัลบั้ม /
อัปโหลดส่งคลาวด์...</Text>
            </View>
     ) : (
            <TouchableOpacity
              style={[styles.actionTriggerButton, !tempImageUri &&
styles.disabledButton]}
              onPress={handleSaveAndSyncAll}
           disabled={!tempImageUri}
            >
              <Text style={styles.actionTriggerText}>💾 กดรันบันทึก 3 ระบบพร้อมกัน
</Text>
            </TouchableOpacity>
     )}

          {/* รายงานสถานะ Log การบันทึก */}
          <View style={styles.logBox}>
            <Text style={styles.logHeader}>📌 บันทึกผลสำเร็จ (Storage
Logs):</Text>
            <View style={styles.logLine}>
              <Text style={styles.logTitle}>1. Local Sandbox
(Private):</Text>
              <Text style={styles.logValue} numberOfLines={2}>
                {sandboxPath ? sandboxPath : 'ยังไม่ได้จัดเก็บลงดิสก์เอกสารแอป'}
              </Text>
            </View>
```

```
            <View style={styles.logLine}>
              <Text style={styles.logTitle}>2. Shared Library
(Public):</Text>
              <Text style={styles.logValue}>
                {sharedGalleryStatus ? sharedGalleryStatus : 'ยังไม่ได้ส่งเขียนแกล
เลอรีกล้องหลัก'}
              </Text>
            </View>
            <View style={styles.logLine}>
              <Text style={styles.logTitle}>3. Cloud Storage
(Supabase):</Text>
              <Text style={styles.logValueCloud} numberOfLines={2}>
                {cloudUrl ? cloudUrl : 'ยังไม่ได้ส่งอัปโหลดขึ้นเครือข่าย'}
              </Text>
            </View>
          </View>

          {/* ปุ่มลบประวัติ */}
          {(sandboxPath || cloudUrl || tempImageUri) && (
            <TouchableOpacity style={styles.clearButton}
onPress={handleClearEverything}>
              <Text style={styles.clearButtonText}>🗑️ ล้างภาพและไฟล์ Sandbox
ทั้งหมด</Text>
            </TouchableOpacity>
      )}
        </View>

      </ScrollView>

      {/* โมดูลเต็มจอสำหรับเปิดกล้องถ่ายภาพ */}
      <Modal visible={isCameraVisible} animationType="slide"
transparent={false}>
        <View style={styles.cameraContainer}>
          <CameraView
            ref={cameraRef}
            style={styles.cameraViewport}
            facing={facing}
          >
            {/* ส่วนควบคุมกล้องด้านบน */}
            <View style={styles.cameraHeader}>
              <TouchableOpacity
                style={styles.cameraCircleBtn}
                onPress={() => setIsCameraVisible(false)}
              >
                <Text style={styles.cameraCloseIcon}>✕</Text>
              </TouchableOpacity>
              <TouchableOpacity
                style={styles.cameraCircleBtn}
                onPress={() => setFacing(curr => curr === 'back' ? 'front'
```

```
: 'back')}
              >
                <Text style={styles.cameraCloseIcon}>🔄</Text>
            </TouchableOpacity>
              </View>

              {/* ส่วนล่างชัตเตอร์ด้านล่าง */}
              <View style={styles.cameraFooter}>
                <TouchableOpacity
                  style={styles.shutterRing}
                  onPress={handleCapturePhoto}
                >
                  <View style={styles.shutterDot} />
                </TouchableOpacity>
                <Text style={styles.cameraTip}>เลนส์พรีวิวสด: {facing === 'front' ?
'กล้องหน้า' : 'กล้องหลัง'}</Text>
              </View>
            </CameraView>
          </View>
        </Modal>
      </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#0f172a',
  },
  scrollContainer: {
    padding: 20,
  },
  header: {
    alignItems: 'center',
    marginVertical: 10,
  },
  headerTitle: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  headerSubtitle: {
    fontSize: 12,
    color: '#64748b',
    textAlign: 'center',
    marginTop: 6,
    lineHeight: 18,
  },
  avatarCard: {
```

```
  backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 20,
    alignItems: 'center',
    borderWidth: 1,
    borderColor: '#334155',
    marginVertical: 15,
  },
  cardLabel: {
    color: '#94a3b8',
    fontSize: 13,
    fontWeight: '600',
    marginBottom: 12,
  },
  avatarWrapper: {
    width: 150,
    height: 150,
    borderRadius: 75,
    borderWidth: 4,
    borderColor: '#38bdf8',
    justifyContent: 'center',
    alignItems: 'center',
    overflow: 'hidden',
    backgroundColor: '#0f172a',
  },
  avatarImage: {
    width: '100%',
    height: '100%',
    resizeMode: 'cover',
  },
  avatarPlaceholder: {
    width: '100%',
    height: '100%',
    justifyContent: 'center',
    alignItems: 'center',
  },
  placeholderText: {
    color: '#475569',
    fontWeight: 'bold',
    fontSize: 16,
  },
  imageSourceIndicator: {
    color: '#10b981',
    fontSize: 12,
    fontWeight: '600',
```

```
    marginTop: 12,
  },
  buttonGroupCard: {
    backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 16,
   borderWidth: 1,
    borderColor: '#334155',
    marginBottom: 15,
  },
  cardHeaderTitle: {
    color: '#ffffff',
    fontSize: 14,
    fontWeight: 'bold',
    marginBottom: 12,
  },
  rowButtons: {
    flexDirection: 'row',
    justifyContent: 'space-between',
  },
  primaryButton: {
    flex: 0.48,
    backgroundColor: '#0284c7',
    borderRadius: 10,
    height: 44,
    justifyContent: 'center',
    alignItems: 'center',
  },
  darkBlueButton: {
    backgroundColor: '#334155',
  },
  buttonText: {
   color: '#ffffff',
    fontWeight: 'bold',
    fontSize: 13,
  },
  actionCard: {
    backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 16,
    borderWidth: 1,
    borderColor: '#334155',
    marginBottom: 20,
  },
  actionTriggerButton: {
    backgroundColor: '#10b981',
```

```
    borderRadius: 10,
    height: 48,
    justifyContent: 'center',
    alignItems: 'center',
  },
  disabledButton: {
    backgroundColor: '#475569',
    opacity: 0.5,
  },
  actionTriggerText: {
    color: '#ffffff',
  fontWeight: 'bold',
    fontSize: 14,
  },
  spinnerBlock: {
    alignItems: 'center',
    marginVertical: 10,
  },
  processingText: {
    color: '#94a3b8',
    fontSize: 11,
    marginTop: 6,
  },
  logBox: {
    backgroundColor: '#0f172a',
    borderRadius: 10,
    padding: 12,
    borderWidth: 1,
    borderColor: '#334155',
    marginTop: 15,
  },
  logHeader: {
    color: '#f8fafc',
    fontSize: 12,
    fontWeight: 'bold',
    marginBottom: 8,
  },
  logLine: {
    marginBottom: 10,
  },
  logTitle: {
    color: '#38bdf8',
    fontSize: 11,
    fontWeight: '600',
  },
  logValue: {
```

```
    color: '#e2e8f0',
    fontSize: 10,
    fontFamily: 'monospace',
    marginTop: 2,
  },
  logValueCloud: {
    color: '#a7f3d0',
    fontSize: 10,
    fontFamily: 'monospace',
    marginTop: 2,
  },
  clearButton: {
    borderWidth: 1,
    borderColor: '#ef4444',
    borderRadius: 10,
    height: 40,
    justifyContent: 'center',
    alignItems: 'center',
    marginTop: 15,
  },
  clearButtonText: {
    color: '#ef4444',
    fontSize: 12,
    fontWeight: 'bold',
  },
// ส่วนประกอบกล้อง Modal
  cameraContainer: {
    flex: 1,
    backgroundColor: '#000000',
  },
  cameraViewport: {
    flex: 1,
    justifyContent: 'space-between',
  },
  cameraHeader: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    paddingTop: 50,
    paddingHorizontal: 20,
  },
  cameraCircleBtn: {
    width: 42,
    height: 42,
    borderRadius: 21,
    backgroundColor: 'rgba(0,0,0,0.5)',
    justifyContent: 'center',
```

```
    alignItems: 'center',
  },
  cameraCloseIcon: {
    color: '#ffffff',
    fontSize: 16,
    fontWeight: 'bold',
  },
  cameraFooter: {
    alignItems: 'center',
    paddingBottom: 40,
    backgroundColor: 'rgba(0,0,0,0.3)',
    paddingTop: 15,
  },
  shutterRing: {
    width: 74,
    height: 74,
    borderRadius: 37,
    borderWidth: 4,
    borderColor: '#ffffff',
    justifyContent: 'center',
    alignItems: 'center',
    marginBottom: 12,
  },
  shutterDot: {
    width: 58,
    height: 58,
    borderRadius: 29,
    backgroundColor: '#ffffff',
  },
  cameraTip: {
    color: '#cbd5e1',
    fontSize: 11,
    fontWeight: '500',
  },
});
```

เอกสารอ้างอิง

Expo. (2024). *Camera*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/camera/

Expo. (2024). *FileSystem*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/filesystem/

Expo. (2024). *ImagePicker*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/imagepicker/